Formación SQL 360°

Session 4 : Vistas y Optimización

Facilitador: Flavio Bevilacqua

https://www.linkedin.com/flabevy

Temario de Clase:

- Estructuras de Control
- Bucle While
- Variables
- Vistas
- Índices y Optimización

Estructuras de Control

ESTRUCTURAS DE CONTROL

Las estructuras de control permiten que tu código SQL deje de ser una lista estática de instrucciones y se convierta en un **proceso dinámico** que reacciona según los datos que encuentra.

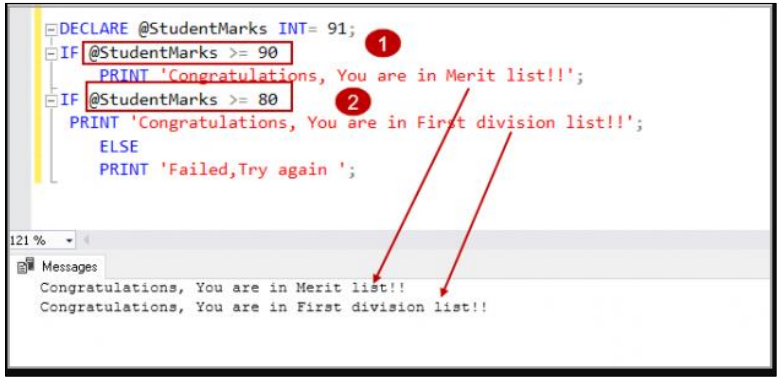
¿Para qué sirven en Negocio?

- **Validaciones:** "Si el stock es menor a 10, genera una alerta; si no, continúa".
- **Segmentación Dinámica:** Aplicar descuentos diferentes según el perfil del cliente en tiempo real.
- **Flujos de Carga:** Ejecutar reportes solo si la tabla de origen se actualizó hoy.

COMANDOS DE DECISIÓN

En SQL Server, evaluamos condiciones lógicas para bifurcar el camino de nuestra ejecución.

- **IF (Si...):** Es la condición inicial. Evalúa si algo es verdadero (ej. ¿Ventas > Meta?).
- **ELSE (De lo contrario...):** Es el plan de contingencia. Se ejecuta automáticamente si la condición del IF no se cumplió.
- **BEGIN...END:** Son los "paréntesis" del bloque. Se usan cuando quieres que varias líneas de código se ejecuten bajo una misma condición.



BUCLE WHILE

BUCLE WHILE

Un bucle **WHILE** permite que un bloque de código se repita **mientras** una condición específica sea verdadera. Es la herramienta ideal para procesos que requieren iteración.

¿Cuándo implementarlo en Negocio?

- **Generación de Calendarios:** Crear automáticamente todas las fechas de un año fiscal.
- **Cargas Masivas:** Procesar datos en "lotes" pequeños (batches) para no bloquear el servidor.
- **Simulaciones:** Ejecutar un cálculo financiero mes a mes hasta llegar a una meta de ahorro.

¡Cuidado con el Bucle Infinito! Si olvidas actualizar la condición, la acción se repetirá **para siempre**, consumiendo todos los recursos del servidor. Siempre debe haber una "salida" clara.

LA REGLA DE ORO!

Para que un bucle sea útil y no un problema, debe cumplir tres pasos críticos:

- **Inicio:** Definir dónde empezamos ***(ej. @Contador = 1).***
- **Condición:** Definir hasta dónde llegamos ***(ej. WHILE @Contador <= 12).***
- **Actualización (Crítico):** Debemos cambiar el valor dentro del bucle ***(ej. @Contador = @Contador + 1).***

VARIABLES

VARIABLES

Una variable es una **"caja con etiqueta"** en la memoria del servidor que guarda un valor temporal para usarlo más adelante en tus cálculos o reportes.

La Metáfora de la Caja:

- **Identidad:** La caja tiene un nombre (ej. @TotalVentas).
- **Contenido:** El valor dentro puede cambiar (**mutar**) a lo largo del programa.
- **Restricción:** El **tipo de objeto** que guarda la caja no cambia. Si la caja es para "Números", nunca podrá guardar "Texto".

DECLARAR VS ASIGNAR

A diferencia de lenguajes como **Python**, donde solo escribes x = 10 y el programa lo entiende, SQL Server es más estricto por seguridad:

- **DECLARE (La Creación):** Reservas el espacio y defines el **Tipo de Dato** (ej. INT, DECIMAL, VARCHAR). Es como decidir el tamaño y material de la caja antes de usarla.
- **SET o SELECT (La Inicialización):** Es el acto de meter el primer valor dentro de la caja.

Característica	SQL Server (T-SQL)	Python
Declaración	**Obligatoria** (DECLARE)	No necesaria
Tipo de Dato	**Estático** (No cambia)	Dinámico (Cambia libremente)
Seguridad	Alta (Evita errores de mezcla de datos)	Flexible (Más rápido de escribir)

VISTAS

VISTAS

Una Vista es una consulta SELECT a la que le asignamos un **nombre propio** y guardamos de forma permanente en la base de datos. Es un objeto identificable (como una tabla real) que actúa como un filtro o resumen predefinido.

- **Naturaleza Dinámica:** No almacena datos físicamente; se actualiza automáticamente cada vez que la información en las tablas originales cambia.
- **Conectividad Externa:** Son el puente ideal para herramientas como **Power BI o Excel**, ya que permiten exponer datos limpios sin que el usuario final vea la complejidad del código.

Diferencias Estratégicas:

- **CTE / Subconsulta:** Existen solo mientras se ejecuta la consulta (son efímeras).
- **Tabla Temporal:** Existe solo durante tu sesión de trabajo.
- **Vista:** Es un **objeto permanente** disponible para todos los usuarios con permisos.

CONSIDERACIONES

- **Nombres Obligatorios:** Todos los campos resultantes deben tener un nombre (alias). No puedes tener columnas sin nombre o duplicadas (ej. resultados de un SUM sin un AS Total).

- **Seguridad:** Permiten mostrar a un usuario solo las columnas que necesita ver, ocultando datos sensibles de la tabla origen.

- **Simplicidad:** Transforman una consulta de 100 líneas en un simple SELECT * FROM Mi_Vista.

INDICES

INDICES

Un **Índice** es una estructura que permite al servidor encontrar filas específicas de forma mucho más rápida. Sin un índice, SQL tiene que leer toda la tabla (página por página) para encontrar un dato; con un índice, va directo a la "dirección" correcta.

La Metáfora del Libro: Imagina buscar un concepto en un libro de 1,000 páginas.

- **Sin Índice:** Tienes que leer hoja por hoja (**Table Scan**).
- **Con Índice:** Vas al final del libro, buscas la palabra y saltas a la página exacta (**Index Seek**).

Impacto en Negocio: Reducción drástica en los tiempos de espera de reportes y menor consumo de recursos del servidor.

Sintaxis: CREATE INDEX ‘Nombre_Indice’(tabla y campo donde se construye el índice.)

INDICES

Imagina que tu tabla de Ventas tiene millones de registros y siempre filtras por ID_Cliente. Sin un índice, SQL busca uno por uno. Con esta instrucción, creas el "diccionario":

SQL

```sql
-- Creamos un índice para acelerar las búsquedas por Cliente
CREATE INDEX idx_Ventas_Cliente
ON Facturacion.Ventas (ID_Cliente);
```

¿COMO SABER SI MI CONSULTA ES EFICIENTE?

En otros motores de base de datos se usa **EXPLAIN ANALYZE**, pero en **SQL Server** utilizamos el Plan de Ejecución Estimado/Real.

- **Plan de Ejecución:** Es el mapa de ruta que el servidor decide seguir para obtener tus datos.
- **Comando Equivalente:** No es un comando de texto como tal, sino un botón en la barra de herramientas ***(o SET STATISTICS TIME ON)*** que te muestra gráficamente dónde está el "cuello de botella".

Tips de Implementación:

- **No indexar todo:** Los índices aceleran las **lecturas** (SELECT), pero pueden ralentizar las **escrituras** (INSERT/UPDATE), ya que el índice debe actualizarse también.
- **Columnas Clave:** Indexa siempre las columnas que usas frecuentemente en tus WHERE y en tus JOIN.

¿COMO SABER SI MI CONSULTA ES EFICIENTE?

Para saber si nuestro índice realmente funcionó, necesitamos que SQL nos diga cuánto tardó en "pensar" y ejecutar la consulta. En SQL Server, activamos el cronómetro interno con este comando:

```sql
-- 1. Activamos el cronómetro
SET STATISTICS TIME ON;

-- 2. Ejecutamos nuestra consulta de negocio
SELECT SUM(Monto)
FROM Facturacion.Ventas
WHERE ID_Cliente = 505;

-- 3. (Opcional) Apagamos el cronómetro al terminar
SET STATISTICS TIME OFF;
```

Live Coding!

Gracias